

Experiment No – 1

The objective of this experiment is to study IC datasheets, verify De Morgan's theorem, realize and simplify a logic function using a K-map, and implement logic functions using NAND/NOR gate ICs, validating results with truth tables.

Caution: Read the datasheet of ICs (SN74LS00N, SN74LS04N, SN74LS08N, SN74LS32N, SN74LS02N) carefully before attempting Lab experiments, as it provides essential information about a component specifications, pin configurations, operating conditions, and performance parameters.

Part-1: Implement and verify De Morgan's theorem using three input variables on a breadboard with 1 K Ω resistance, LEDs, and logic gate ICs (NOT, AND, and OR).

$$(A + B + C)' = A' \cdot B' \cdot C'$$

$$(A \cdot B \cdot C)' = A' + B' + C'$$

Part-2: Realize the Logic function given below using K-map, find the Boolean expression. Implement and verify the logic expression.

$$F(A, B, C, D) = \sum (0, 2, 4, 6, 8)$$

Part-3: Implement the function given below using Breadboard, 1 K Ω resistance, LED, and 2-input NAND/NOR gate ICs and verify using truth table.

$$F = (A + B) \cdot C$$

Experiment No-2

Object: Implementation of BCD to Excess-3 Code Converter using NOT and NAND Gate only.

Experiment – 3

The objective of this experiment is to implement 4x4-bit multiplier as shown in Fig. 1 using breadboard with 1 k Ω resistance, LEDs, three 4-bit adder (SN74LS83), and four AND logic gate (SN74LS08).

$$A = a_3a_2a_1a_0 \text{ (a 4-bit binary number)}$$

$$B = b_3b_2b_1b_0 \text{ (a 4-bit binary number)}$$

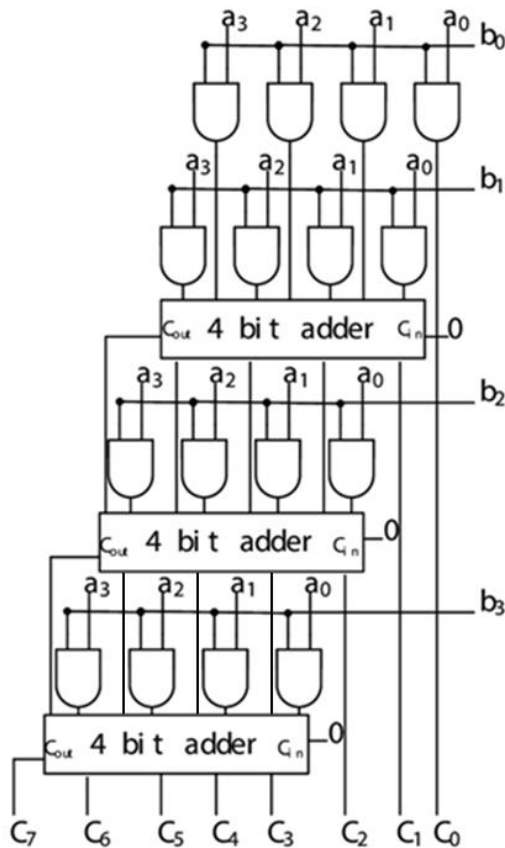


Fig. 1. Logic circuit design of 4x4-bit multiplier

Caution: Read the datasheet of ICs carefully before attempting Lab experiments, as it provides essential information about a component specifications, pin configurations, operating conditions, and performance parameters.

Part-1: Implement and verify 9x8 multiplication using 4x4-bit multiplier.

Part-2: Implement and verify -2x3 multiplication using 4x4-bit multiplier.

Digital Design Lab Manual

Experiment 4

Title

Implementation and Verification of Digital Circuits using Decoder and Multiplexer

Objectives

- **Part A:** Implement and verify the sum of two 2-bit binary numbers using a 3:8 Decoder.
- **Part B:** Implement and verify a **4-input XOR Gate** using a 8:1 Multiplexer.

Theory

Part A: Sum of Two 2-bit Binary Numbers using a Decoder

A **Decoder** is a combinational circuit that converts binary inputs into a unique output. By utilizing appropriate output lines, the sum of two 2-bit binary numbers can be implemented.

Part B: 4-Input XOR Gate using a Multiplexer

A **Multiplexer (MUX)** selects one of several inputs based on control signals. A **4-input XOR gate** can be implemented using a **8:1 MUX**, following the logic equation:

$$Y = A \oplus B \oplus C \oplus D$$

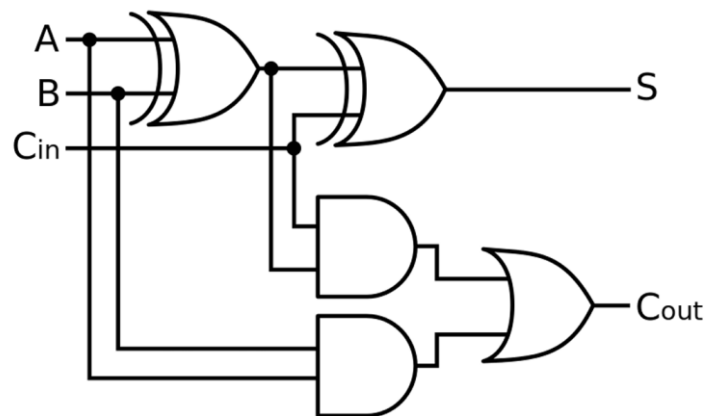
Precaution

Read the datasheet of ICs carefully before attempting lab experiments, as it provides essential information about specifications, pin configurations, and operating conditions.

Experiment – 5

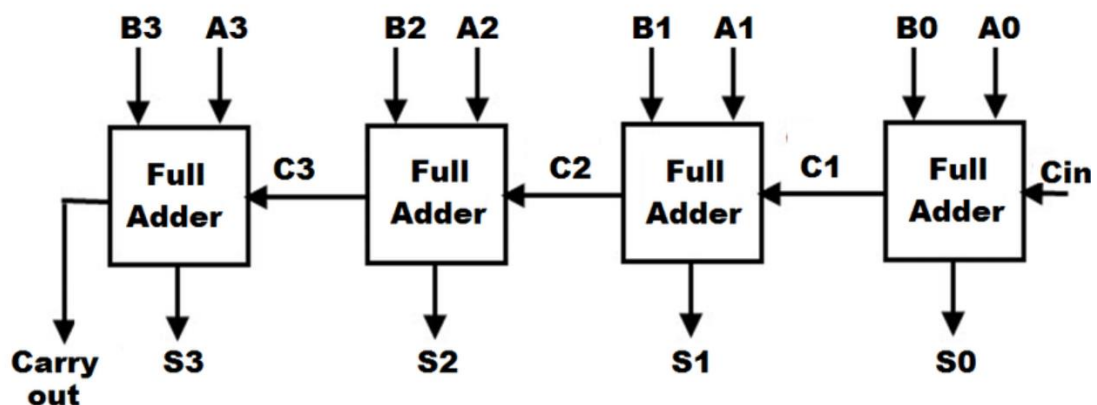
Part-1: To design a 1 bit full adder using Boolean expression in Verilog.

Circuit Diagram:



Part-2: To design and implement a 4-bit binary adder using full adders and logic gates in Verilog.

Circuit Diagram



Experiment-6

1. Using a case statement, write an HDL behavioural description of an 8-bit arithmetic logic unit (ALU). The circuit has a 3-bit select bus (sel), 8-bit input datapaths (A&B), an 8-bit output datapath (y) and performs the arithmetic and logic operations listed below:

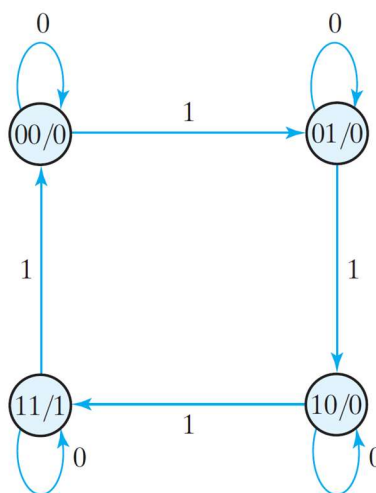
Sel	Description
000	Reset y to all 0's
001	Bitwise AND
010	Bitwise OR
011	Add (Assume A and B are unsigned)
100	Subtract
101	Bitwise exclusive-OR
110	Bitwise complement
111	Set y to all 1's

2. In above given question (1) with all same data modify the 8-bit ALU and write an HDL description so that it has 3-state output controlled by an enable input, En.

3. Write a Verilog model of a circuit whose 32-bit output is formed by shifting its 32-bit input three positions to the right and filling the vacant positions with the bit that was in the MSN (Most Significant Number) before the shift occurred (shift arithmetic right). Write a Verilog model of a circuit whose 32-bit output is formed by shifting its 32-bit input three positions to the left and filling the vacant positions with 0 (shift logical left).

Experiment-7

1. Implement an SR latch using two NOR gates.
 - a. Provide the forbidden input and observe the output.
 - b. Observe the outputs corresponding to complementary inputs i.e. (0,1) and (1,0).
 - c. Check that for input (0,0), it randomly provides different outputs as (0,1) and (1,0). You may try it by switching the power of DC supply ON and OFF multiple times.
 - d. Now, repeat (a) and change the inputs as (0,0) and observe the output to make sure whether it retains the previous state. You may repeat it also multiple times by turning the DC supply ON and OFF. Observe the difference w.r.t to (c).
2. Design a D latch by adding the Enable option in the above designed SR latch.
 - a. Design a D Flip Flop by using Master and Slave D latches.
 - b. Provide a 400mHz square wave as clock. Use a buffer by two NOT gates to introduce a delay and use the resultant signal as input D of the Flip Flop.
 - c. Observe the outputs by LEDs and increase the frequency of the clock. Observe the maximum frequency till which the switching is visible by your eyes.
3. Implement a T Flip Flop using the already designed D Flip Flop.
 - a. Implement a binary counter using T Flip Flops. The state diagram of the counter is following:



Experiment-8

- Design a 4-bit (MOD-16) synchronous up counter using JK (IC: - SN74LS76N) flip flop. Which should count from 0 to 15 (0000 to 1111) and start again from 0 (0000). Use function generator to provide 400mHz clock frequency and observe the output on 7-segment display using BCD to 7-segment decoder IC(SN74LS47N).
 - Modify the above designed circuit to make it MOD-10 up counter which should count from 0 to 9(0000 to 1001) and start again from 0(0000). Display the output on 7-segment display.
- Using two 4-bit universal shift register (SN74LS194AN) implement the circuit shown in figure1. Load register A and B with some initial 4-bit value and perform the right shift operation of 1-bit for each clock cycle and observe the changes in register value through LEDs.

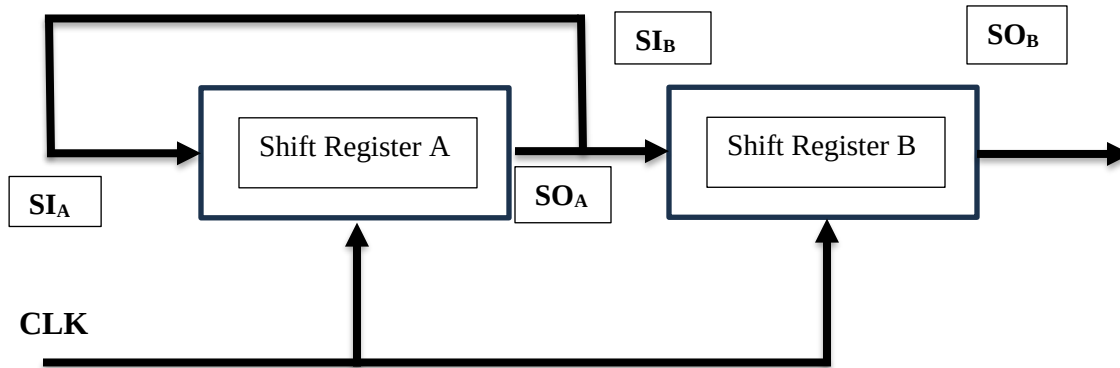


Figure1.

Note: - Please read the IC datasheet and manuals carefully.

Experiment - 09

Design and implement a decade counter that counts from **00 to 99** using the **74LS93** IC, a **4-bit binary counter**.

1. After reaching 9 (1001 in binary), the counter should reset and restart from **0**.
2. The binary output from the **74LS93** needs to be converted to decimal using the **7447** IC, a **BCD to 7-segment decoder**.
3. The decoded output will be displayed on a **7-segment display**.

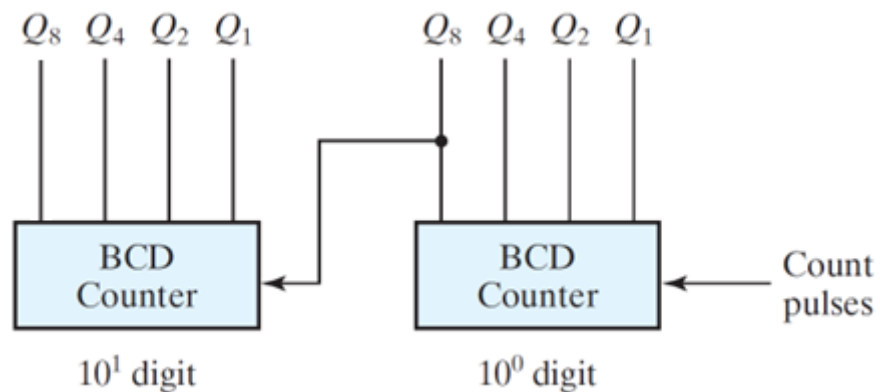


Fig. 1. Decade counter

Experiment – 10

The objective of this experiment is to implement 4-bit adder and traffic light control on FPGA (PYNQ-Z2) board as shown in Fig. 1.

Caution:

1. Read the user manual of PYNQ-Z2 board carefully before attempting lab experiments, as it provides essential information about input port, out port, add-on devices and others.
https://www.mouser.com/datasheet/2/744/pynqz2_user_manual_v1_0-1525725.pdf?srltid=AfmBOopEUgo94iFMiboTJ4GR96gFh4pUiQe8DL8UrHUc73b4ua8GmDO0
2. Download the master XDC and Add-on board file from the given link that helps to implement the design on the PYNQ-Z2 board.
<http://www.tul.com.tw/download/pynq-z2.zip>
http://www.tul.com.tw/download/pynq-z2_v1.0.xdc.zip

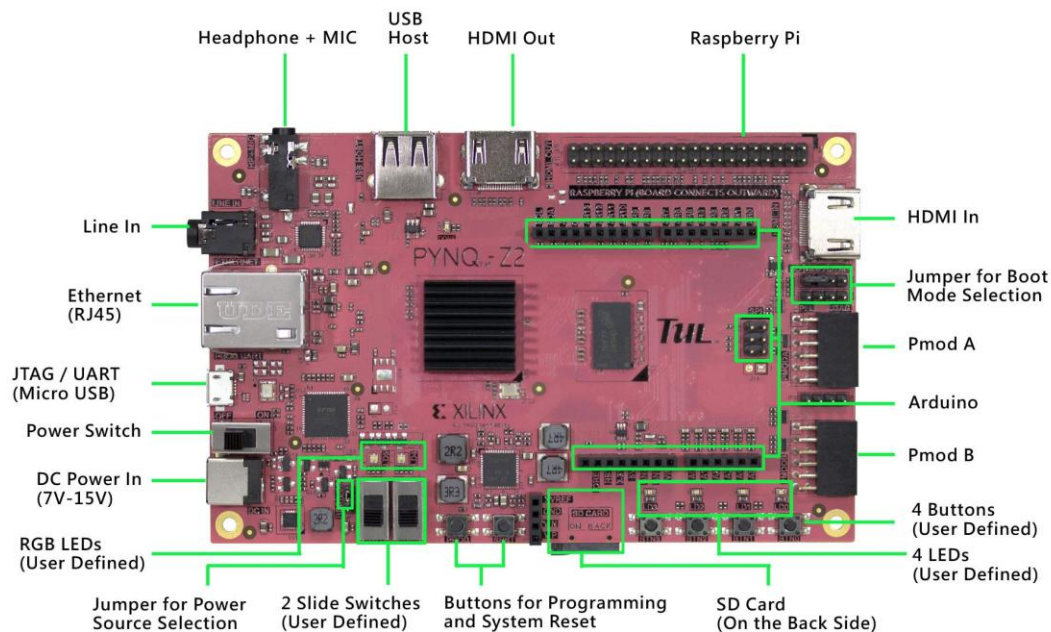


Fig. 1. FPGA (PYNQ-Z2) board.

Part-1: Implement the 4-bit binary addition on PYNQ-Z2 (Use previous simulation code).

Part-2: Implement and verify traffic light control on PYNQ-Z2.

1. Write the code for traffic light control and check your design using RTL simulation.
2. Run synthesis, implementation, generate bit-stream, and program your device.